RABIT-KV: A PHYSICALLY PACKED, TARGET-BIT-AWARE KV CACHE FOR LLM SERVING

Ryan Zhang

ABSTRACT

Low-bit KV-cache quantization is often described by its logical precision, but a serving system benefits only when that bit budget becomes allocator capacity. Between the two lie costs that a nominal bit-width omits: quantization metadata, full-precision residual tokens, partially filled groups, and the online aging of cache state. We present RABIT-KV, a target-bit-aware, physically packed KV-cache system that jointly accounts for asymmetric key/value precision, residual state, metadata, online cache aging, and allocator capacity, integrated into the vLLM serving engine. On the evaluated H100 setup with Llama-3.1-8B, RABIT-KV's paged block pool holds $5.28\times$ the KV tokens of BF16, $2.64\times$ those of FP8, and $1.72\times$ those of the tested TurboQuant configuration, at $+8.33\%$ median single-request time per output token relative to that configuration; the residual and open-group state of each sequence is held outside the pool and counted separately. The comparison with FP8 and TurboQuant covers capacity and single-request latency only. Against BF16, Llama continuation perplexity rises by 1.56%, NIAH and Passage Retrieval retain their tested scores, and HotpotQA is approximately 1 F1 lower with a confidence interval spanning zero; Qwen2.5-7B, at the same operating point, shows severe model-specific quality sensitivity. The current correctness-first implementation does not convert its capacity advantage into higher throughput on the evaluated workloads. Physical capacity, quality, and serving cost must therefore be evaluated jointly.

1 INTRODUCTION

Once model weights are loaded, the key–value (KV) cache is a major GPU-memory consumer in LLM serving and can become dominant at long context lengths or high concurrency. It grows linearly with context length and with the number of in-flight requests, and a paged allocator (Kwon et al., 2023) admits a request only when blocks are available for it. KV memory therefore bounds how much context and how many concurrent requests one GPU can hold. Quantizing the cache is a direct response (Liu et al., 2024; Hooper et al., 2024; Zandieh et al., 2025), and such methods are often characterized by a logical bit-width: the number of bits per stored key or value element. The quantity a serving system can use, however, is physical: the number of tokens its allocator can place in a fixed memory budget. The two are related but not interchangeable.

A nominal low-bit cache falls short of its bit budget for reasons that are individually well known. Keys and values tolerate different precisions (Liu et al., 2024; Tao et al., 2024). Grouped affine quantization stores a minimum and a scale per group, and at two or three bits per element this metadata is a substantial fraction of the payload. The newest tokens are commonly kept at full precision (Liu et al., 2024; Duanmu et al., 2024). Groups are partially filled while a sequence is being generated, and cache state must be re-encoded online as tokens age from one representation to the next. The result must then fit the fixed-size blocks of the engine's allocator and be read by attention kernels at an acceptable cost. Our focus is the systems problem of accounting for these effects jointly and measuring the resulting allocator-visible capacity and runtime cost.

RABIT-KV is a target-bit-aware, physically packed KV cache. One fixed operating point combines 3-bit keys, 2-bit values, 32-element groups, a four-token full-precision residual window, and 8-bit quantized group metadata, with an online lifecycle in which tokens age from the residual window into an open group and then into closed, packed pages. The "2-bit target" denotes this operating point rather than a literal end-to-end storage cost. The pages are stored in the paged allocator of vLLM (Kwon et al., 2023), and capacity is reported as the number of tokens the engine allocates in its paged block pool. On one H100 with Llama-3.1-8B, RABIT-KV's pool holds $5.28\times$ the KV tokens of BF16, $2.64\times$ those of FP8, and $1.72\times$ those of the tested TurboQuant configuration (Zandieh et al., 2025; vLLM contributors, 2026), at $+8.33\%$ median time per output token relative to that configuration in a matched single-request session. The residual window and open group of each sequence, up to 3 MiB per sequence, lie outside the pool and are accounted for separately. The four-way comparison covers capacity and single-request latency; context scal-

Preprint. Code, data, and evidence: https://github.com/ryanz090504-star/RABIT-KV.

ing, concurrency, and quality are compared with BF16 only. Quality loss on Llama-3.1-8B is small (continuation perplexity $+1.56\%$), whereas Qwen2.5-7B shows severe sensitivity at the same operating point; we therefore do not assume that quality transfers across models.

The broader lesson is that aggressive KV compression should be judged as a joint trade-off among physical capacity, quality, and serving cost, not by nominal bits alone. RABIT-KV makes the cost side explicit. Its capacity raises the admissible load: with 8192-token prompts, BF16 reaches its allocator ceiling at 47 in-flight requests, whereas RABIT-KV sustains 64. Yet in every concurrency workload we tested, RABIT-KV's throughput remains below that of BF16, and prompts that extend beyond the first 16,384-token prefill chunk incur a large time-to-first-token penalty. The current implementation prioritizes correctness and physical packing over cross-request and cross-token kernel batching, so a capacity gain does not by itself yield an end-to-end serving benefit. We report these costs alongside the capacity result and leave a batched runtime to future work.

Contributions.

- **Design.** A target-bit-aware KV-cache representation with an online cache lifecycle covering asymmetric key/value precision, full-precision residual state, quantized group metadata, open groups, closed pages, and cache aging. Rather than treating these mechanisms independently, RABIT-KV integrates them into one lifecycle whose target bit budget maps to a concrete physical layout.

- **Physical system.** A physically packed integration into the vLLM serving engine, whose benefit is measured as allocator-observed capacity of the paged block pool rather than by logical compression accounting: $5.28\times$ BF16, $2.64\times$ FP8, and $1.72\times$ the tested TurboQuant configuration on Llama-3.1-8B, with the same ratio over BF16 on a second model geometry; per-sequence state outside the pool is accounted for separately.

- **Evaluation.** A systems evaluation that compares BF16, FP8, TurboQuant, and RABIT-KV on capacity and single-request latency, and RABIT-KV with BF16 on context scaling, concurrency, and quality (Llama-3.1-8B and Qwen2.5-7B, with an evaluator verified against an independent reference). It exposes the capacity gain together with its latency, throughput, and model-dependent quality trade-offs; the comparison with FP8 and TurboQuant is not quality-matched.

2 Background and Motivation

KV-cache memory. For every token, each decoder layer stores one key and one value vector per KV head. With H KV heads of dimension d, a token therefore holds $2Hd$ elements per layer. For Llama-3.1-8B ($H = 8$, $d = 128$, 32 layers) this is 4,096 bytes per token per layer in BF16, or 128 KiB per token. The state lives as long as its request, so KV memory grows with the sum of the context lengths of the requests in flight.

Paged allocation. Serving engines such as vLLM (Kwon et al., 2023) manage this memory as a pool of fixed-size blocks. A block holds the KV state of a fixed number of consecutive tokens (32 in our configuration), and a per-request block table maps token positions to blocks. The pool is sized once, at start-up: the number of blocks is the KV memory budget divided by the bytes that one block occupies. A request is admitted, and can grow, only while free blocks remain. Throughout this paper, *capacity* means the number of blocks times the tokens per block as reported by the running engine. It is the quantity the scheduler can use.

Logical precision is not physical capacity. Quantized KV caches (Liu et al., 2024; Hooper et al., 2024; Zandieh et al., 2025) are often described by the bit-width of their codes, although some work reports metadata-inclusive average bits (Duanmu et al., 2024) or peak memory (Liu et al., 2024). The bytes a block occupies depend on more than the code width. First, affine quantization stores a minimum and a scale per group. With groups of 32 elements, two 16-bit parameters add one bit per element, which is a third of a 3-bit payload and half of a 2-bit one. Second, some state is kept at full precision: recent tokens in residual-window designs, or entire layers, as in the TurboQuant configuration we test, which leaves four boundary layers in BF16. Third, when quantization statistics span several tokens, the group at the end of a growing sequence is incomplete and has no final encoding yet. Whether each of these costs is stored inside the block pool, beside it, or not at all determines how many blocks fit in the budget. A bit-width alone therefore does not determine capacity; it has to be measured in the engine, together with the runtime cost of reading the packed state.

3 RABIT-KV Design

RABIT-KV is defined by a single operating point, written K3 / V2 / G32 / R4 / META8g64. It was fixed during earlier development and was not retuned for any experiment in this paper. We describe it as one configuration, not as the best member of a tuned family, and we do not claim that each component is individually optimal. Its ingredients follow

prior representations. Quantizing keys per channel along the token axis and values per token follows KIVI (Liu et al., 2024), KVQuant (Hooper et al., 2024), and AsymKV (Tao et al., 2024). KIVI and AsymKV use groups of 32, which RABIT-KV also adopts. A window of recent full-precision tokens appears in KIVI and SKVQ (Duanmu et al., 2024), unequal key and value precision in AsymKV and SKVQ, and reduced-precision quantization parameters counted in the effective bit-width in SKVQ. What this section specifies is how these mechanisms are combined into one online lifecycle with a fixed physical layout.

3.1 Operating point

K3: keys. Keys are stored as 3-bit affine codes grouped along the *sequence* axis. For each (head, channel) pair, 32 consecutive tokens form a group with one minimum and one scale, $s = (\max - \min)/7$.

V2: values. Values are stored as 2-bit affine codes grouped along the *channel* axis. For each (token, head) pair, 32 consecutive channels form a group with one minimum and one scale, $s = (\max - \min)/3$.

G32: group size. Both groupings use 32 elements. Thirty-two tokens are also the unit of physical storage: one K group along the sequence, the values of the same 32 tokens, and their metadata form one page, and one page occupies one allocator block.

R4: residual window. The newest four tokens of every sequence are kept exactly, in BF16.

META8g64: metadata. The minimum and scale arrays are themselves quantized. Each array is split into groups of 64 values and stored as 8-bit codes with one BF16 minimum and scale per group. Keys and values are dequantized with the parameters decoded from this second level, not with the original ones.

The two axes have different consequences. A value group lies within one token, so a token's value codes and primary minimum and scale can be computed once, as soon as the token leaves the residual window, independently of every other token. A key group spans 32 tokens, so its metadata is amortized over a page, with one minimum and scale per (head, channel), but its statistics are not final until all 32 tokens exist. Keys also receive one more bit than values. These choices define the lifecycle below; their effect on quality is measured only for the operating point as a whole.

3.2 Cache lifecycle

A token passes through three states (Figure 1).

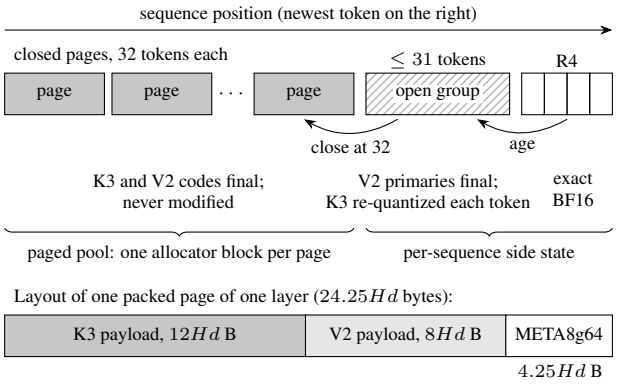


Figure 1. Cache lifecycle and physical layout. A token enters the exact four-token residual window (R4), ages into the open group, and becomes part of a packed page when the group reaches 32 tokens. Closed pages occupy allocator blocks; the open group and the residual window are per-sequence state outside the paged pool. "V2 primaries": value codes and FP32 minimum and scale; the decoded values can still change (Section 3.2). Bottom: byte layout of one page, to scale (H KV heads of dimension d).

Residual. A new token enters the residual window with exact keys and values.

Open group. When appending a token would make the residual window exceed four tokens, the oldest residual token leaves the window and joins the *open group*, the incomplete K group at the end of the sequence. Its 2-bit value codes and FP32 primary minimum and scale are computed now and never change, but its decoded values can: META8g64 is recomputed at each read, and a metadata group spans two tokens on Llama-3.1-8B and four on Qwen2.5-7B, so until that group is full a later token can change them. Its keys cannot be encoded on entry: the open group is quantized as a 32-token group padded with zeros, so the group statistics, and with them the codes of every key already in the group, are recomputed as the group grows. The padding enters only the minimum and maximum; only the real keys are attended. The exact keys of the open group are therefore kept in BF16. Rounding, zero-range handling, the flattening order of the metadata, and the storage of the open group are specified in the supplementary appendix.

Closed page. When the open group reaches 32 tokens it closes. Its key codes and metadata become final, and the key payload, the value payload, and the META8g64 metadata are written as one packed page. A closed page is never modified again.

A sequence of $n \geq 4$ tokens thus consists of $\lfloor (n-4)/32 \rfloor$ closed pages, an open group of $(n-4) \bmod 32$ tokens, and four residual tokens. Once this state exists, attention reads

all three: closed pages and the open group in quantized form, the residual window exactly. (The serving engine computes the initial prefill of a prompt over exact keys and values before it builds this state; Section 4.) Aging is online. It happens as tokens are appended during prefill and decoding, there is no offline compression pass, and the representation of a key keeps changing until its group closes. This is why we validate the lifecycle token by token and not only its final state (Section 5).

3.3 Physical accounting

We distinguish four quantities that are often conflated.

Target-bit label. "2-bit target" names the operating point. It is not a storage cost.

Logical payload bits. A token has equally many key and value elements, so 3-bit keys and 2-bit values give 2.5 payload bits per element.

Physical page bytes. For one layer, a page holds $32Hd$ key elements and $32Hd$ value elements: $12Hd$ bytes of key payload and $8Hd$ bytes of value payload. There are four metadata arrays (minimum and scale, for keys and for values), each with Hd entries per page: one per (head, channel) for keys, and one per (token, head, channel group) for values, which is $32 \cdot H \cdot d/32$. The parameters are computed in FP32; stored directly in BF16, a layout we did not build, they would take $8Hd$ bytes, 40% of the payload. Under META8g64 each array takes Hd bytes of codes plus $Hd/16$ bytes of group parameters, $4.25Hd$ bytes in total. A page therefore occupies $24.25Hd$ bytes per layer. For Llama-3.1-8B ($Hd = 1024$) this is 24,832 bytes per page, or 776 bytes per token per layer, against 4,096 bytes in BF16. Dividing by the $2Hd = 2048$ key and value elements of a token in one layer gives about 3.03 bits per element. This is a derived, closed-page figure and not our headline metric. It counts payload and metadata of closed pages only. It excludes the residual window, the open group, and any rounding by the allocator.

Allocator-observed capacity. The number of tokens the engine can place in its KV budget is the quantity we report as the result. The page bytes above predict a ratio of $4096/776 \approx 5.28$ over BF16. Section 6.1 reports the value measured in the running engine and compares it with FP8 and with the tested TurboQuant configuration. This capacity measures the paged KV pool. The residual window and the open group are per-sequence state of at most 35 tokens, held outside the pool and not counted in it, so the capacity ratio is not a statement about total device memory including all auxiliary state of active sequences.

4 Physical Implementation

Integration. RABIT-KV is implemented as an additional KV-cache type in the Triton attention backend of vLLM (Kwon et al., 2023), next to the FP8 and TurboQuant cache types of our engine snapshot. The scheduler, the block allocator, and the per-request block tables are unchanged. The engine sizes its block pool from the bytes per block, so declaring the packed page size as the block size is what turns the representation into capacity: the same memory budget yields more blocks, and the scheduler admits requests against them as it does for any other cache type. There is no separate compressed pool and no background compaction.

Packed pages and per-sequence state. One allocator block holds one closed page per layer: packed 3-bit key codes, packed 2-bit value codes, and the four META8g64 metadata arrays. A page is written into its scheduler-assigned block at the moment its group closes. The residual window and the open group are kept in a per-request structure outside the pool, of at most 35 tokens per sequence.

Execution paths. Three paths read and write this state. *Initial prefill:* for the first prefill chunk of a request, attention is computed densely over the exact keys and values of the chunk, and the chunk is then encoded in bulk into closed pages, an open group, and a residual window. *Later prefill chunks:* when a prompt exceeds the per-step token budget (16,384 tokens in our configuration), or when that budget is shared among requests, the remaining tokens are processed one query at a time in the causal order of the chunk. Before each query attends, the state is advanced exactly as if its token had been appended, so a page that closes within the chunk becomes visible to the later queries of that chunk; such pages are encoded once, from exact keys, when the chunk starts. In our timing runs, consecutive queries that see the same closed pages (at most 32) share one decode of those pages, while the state update and the open-group and residual work stay per query. *Decode:* each new token is appended, the lifecycle of Section 3 advances by one step, and attention is computed in parts: over the closed pages, which are dequantized inside the kernel and never materialized in full precision, over the open group and the residual window, and a final reduction that combines the partial results.

What is not batched. In the current implementation these paths run once per request, and append, aging, and the open-group work run once per token. Requests in the same scheduler step are processed one after another within each layer, while the rest of the model is batched as usual. The implementation was built for correctness and physical

packing first. Section 6.3 measures the consequence for latency and throughput.

Model geometries. The kernels are specialized for the Llama-3.1-8B attention geometry. Qwen2.5-7B has a different head layout (four KV heads) and is served through exact general fallback paths: the same representation, pages, and capacity accounting, executed by unspecialized code. Its latency therefore measures the fallback paths and is not evidence about how the specialized kernels would transfer.

Correctness. A fixed test suite is run before every serving experiment. It checks the page layout, the page codec against a reference implementation, the online state against that reference at every step, the metadata encoding, grouped-query attention, and causal chunked prefill. These tests establish agreement with the reference on the tested cases; they are not a formal verification.

5 Experimental Methodology

Setup. Every run uses one NVIDIA H100 (80 GB). The engine is a vLLM snapshot (Kwon et al., 2023) with our changes, in eager mode, with 32-token blocks, a GPU memory fraction of 0.82, and a maximum model length of 32,768 tokens. All methods run in eager mode. Because CUDA graph capture can interact differently with attention backends, the reported latency comparisons characterize this eager configuration and need not transfer unchanged to graph-enabled deployments. The models are Llama-3.1-8B-Instruct (Meta, 2024) and Qwen2.5-7B-Instruct (Yang et al., 2024) with BF16 weights; the quality experiments pin and verify the model revisions. Baselines are the BF16 cache, the engine's native per-tensor FP8 cache, and one configuration of the upstream TurboQuant backend (vLLM contributors, 2026), unmodified in our snapshot (3-bit keys, 4-bit values), which quantizes 28 layers and keeps four boundary layers in BF16. TurboQuant runs with its own attention backend, so comparisons with it are between systems as deployed, not between quantization kernels.

Physical capacity. Capacity is read from the running engine as the number of allocated blocks times 32 tokens, under the same memory budget for every method. It is not computed from byte estimates.

Latency and throughput. Absolute latency varies between sessions, so methods are compared only within one session. The primary comparison serves all four methods in a mirrored order (A B C D D C B A), two legs per method and 30 single-request samples per leg, with a 2048-token prompt and 32 greedily decoded output tokens. We report medians of time per output token (TPOT) and time to first token (TTFT); these statistics are descriptive and carry no confidence intervals. Relative differences are computed from unrounded measurements; displayed absolute values are rounded. Context scaling uses single requests with prompts from 512 to 32,736 tokens and 15 samples per cell. Concurrency uses a closed loop at 1 to 64 in-flight requests with 2048- and 8192-token prompts, 256 measured requests per point after unmeasured conditioning, and three trials per point. The supplementary appendix reports dispersion within each session: by leg, by sample within a cell, and by trial, respectively.

Quality evaluator. Quality is measured logically, outside the serving engine, with a cache object for Hugging Face Transformers (Wolf et al., 2019) that implements the representation and lifecycle of Section 3. Serving and logical evaluation use different execution machinery: the evaluator reproduces the stored-cache representation and its aging, not the serving kernels or their schedule. Each evaluation begins with a dense BF16 prefill of a prompt prefix, so those prefilled tokens attend to exact history, as in the initial prefill of the serving engine. The resulting keys and values for that prefix are then converted in one step into the RABIT-KV state for that length: closed pages, an open group, and four residual tokens. Every later token is processed one per forward pass. It is appended, the lifecycle advances, and the token attends to the state at its own position, so no token stays exact beyond the residual window. The BF16 arm runs the same loop on the same inputs with an unmodified cache.

Tasks. Continuation perplexity is measured on WikiText-2 (Merity et al., 2016) for both models, over 32 windows of 1,024 context tokens and 128 scored tokens, all with teacher forcing. The first scored token is predicted from the BF16 prefill and is identical in both arms; the other 127 are predicted from the RABIT-KV state. Long-context quality is measured for Llama-3.1-8B only, with greedy decoding: a synthetic needle-in-a-haystack task of our own construction, in which a secret code is inserted at 19 depths into WikiText-2 filler text (57 cases at 4,096, 8,192, and 16,384 tokens) and the Passage Retrieval (200 examples) and HotpotQA (100 examples) tasks of LongBench (Bai et al., 2024; Yang et al., 2018). Here all but the last prompt token are prefilled and the last prompt token is the first step on the converted state, so every generated token is predicted from RABIT-KV state. Prompts are at most 16,384 tokens and the evaluator never executes a later prefill chunk. The quality of prompt tokens that attend to an already packed prefix, which occurs in serving beyond the per-step token budget, is therefore not measured by the evaluator; one such schedule is measured in the serving engine (Section 6.4). Confidence intervals of the registered evaluator runs are paired percentile bootstraps with 10,000 resamples.

Validation against an independent reference. The state transformation and aging computed by the evaluator were checked bit for bit against an independently written, frozen reference implementation, on the same device. The check covers synthetic states, including token-by-token aging, and the keys and values the models actually produce on the GPU: every layer of both models, and a 16k-token context for Llama-3.1-8B. CPU and GPU arithmetic differ in the last bit of some scales, which moves a small fraction of codes by one level, so conformance is established on the device that runs the evaluation and we define no cross-device tolerance. This check establishes that the evaluator computes the intended stored state. It is not a formal verification, and it says nothing about model quality or serving performance. The serving implementation is tested against the same reference (Section 4); we do not compare serving outputs with evaluator outputs end to end.

Registered protocol. Each quality run is frozen before execution: inputs, example selection, scorers, bootstrap seeds, and the rules for interpreting the outcome. Nothing is tuned in response to results. Validity checks on output quality use only BF16 outputs, and a run that fails a validity check is excluded.

Scope of the quality evidence. Earlier development results were produced with an evaluator that did not implement this lifecycle exactly: it ordered the value metadata differently, and it quantized the prompt once without aging generated tokens. We report quality only from the validated evaluator, never pool the two, and describe the status of the earlier results in the supplementary appendix.

6 EVALUATION

The evaluation answers four questions. **RQ1**: does RABIT-KV turn its representation into allocator-visible capacity? **RQ2**: what serving cost accompanies that capacity, relative to BF16, FP8, and the tested TurboQuant configuration? **RQ3**: how does the current implementation behave as context length and concurrency grow? **RQ4**: what quality does the fixed operating point retain, and does that behavior transfer across models? Our evaluation focuses on the capacity–cost trade-off rather than a speedup claim.

6.1 Physical Capacity

Table 1 reports the capacity each engine configuration allocates under the same memory budget. RABIT-KV allocates 2,074,592 tokens against 393,024 for BF16, 785,568 for FP8, and 1,203,200 for the tested TurboQuant configuration: $5.28\times$, $2.64\times$, and $1.72\times$ their capacity, respectively. The measured ratio over BF16 agrees with the page accounting of Section 3 ($4096/776$), so the packed page, including its metadata, is what the allocator sizes its blocks by. On the second model geometry, Qwen2.5-7B, the engine allocates 902,656 tokens in BF16 and 4,764,640 with RABIT-KV, the same $5.28\times$.

Method	Pool bytes/token[a]	Tokens	vs. BF16
BF16	131,072	393,024	$1.0000\times$
FP8	65,536	785,568	$1.9988\times$
TurboQuant[b]	42,816	1,203,200	$3.0614\times$
RABIT-KV	24,832	2,074,592	$5.2785\times$

Table 1. Allocator-observed KV capacity on one H100 with Llama-3.1-8B, in one matched session with the same KV memory budget (about 48 GiB). Capacity is the number of allocated 32-token blocks times 32 (12,282, 24,549, 37,600, and 64,831 blocks). [a]Paged-pool bytes: what one token's KV state occupies in the block pool over all 32 layers, from each method's block layout. This is storage accounting, not the measured quantity, and it excludes RABIT-KV's per-sequence residual and open-group state, which is held outside the pool. [b]Tested configuration: 28 quantized layers and four boundary layers kept in BF16.

These numbers measure the paged KV pool. The residual window and the open group of each active sequence are held outside it (Section 3), in per-layer buffers of 3 MiB in total per decoding Llama-3.1-8B sequence in the current implementation. Added to the pool blocks of one sequence, from requested buffer sizes and not measured, they lower its ratio over BF16 to $4.70\times$ for 1,024 tokens and $5.20\times$ for 8,192 (supplementary appendix). The ratio is therefore neither a reduction of total device memory nor a statement that $5.28\times$ as many requests can be served under an arbitrary workload; Section 6.3 shows what the capacity does and does not buy.

6.2 Capacity–Latency Trade-off

All four methods were served in one matched session (Section 5); we do not mix absolute timings from different sessions. Median TPOT is 24.19 ms for BF16, 25.75 ms for FP8, 27.79 ms for TurboQuant, and 30.10 ms for RABIT-KV (Figure 2). RABIT-KV is the slowest of the four per output token: $+24.4\%$ relative to BF16, $+16.9\%$ relative to FP8, and $+8.33\%$ relative to TurboQuant. Its median TTFT is also the highest, 144.65 ms against 59.99, 63.43, and 55.03 ms.

Relative to the tested TurboQuant configuration, RABIT-KV provides $1.72\times$ the observed capacity at $+8.33\%$ median single-request TPOT. This statement has a narrow scope. It concerns one TurboQuant configuration, which keeps four boundary layers in BF16 and runs with its own attention backend, so it compares two systems as deployed and does not isolate the efficiency of either quantization kernel. It is a single-request result with two legs per method and no confidence interval, and neither FP8 nor Turbo-

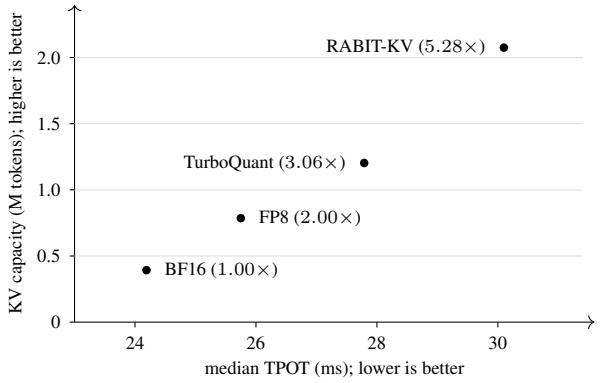


Figure 2. Capacity against per-token latency for the four methods in one matched single-request session (2048-token prompt, 32 output tokens; capacity ratio to BF16 in parentheses). RABIT-KV is the highest-capacity point and the highest-TPOT point. TurboQuant is the tested configuration and runs with its own attention backend.

Measurement	BF16	RABIT-KV
8192-token prompts, closed loop		
In flight at offered C=64	47[a]	64
Throughput at C=16 (req/s)	2.282	1.424
Throughput at C=32 (req/s)	2.346	0.870
2048-token prompts, closed loop		
Throughput at C=8 (req/s)	7.218	1.844
Single request		
TPOT, 16,384 tokens (ms)	15.43	21.15
TTFT, 32,736 tokens (s)	4.4	114.4

Table 2. Scaling behavior of the current implementation (Llama-3.1-8B; medians). [a]BF16 reaches its allocator ceiling and does not attain the offered concurrency in any of three trials. Full curves are in the supplementary appendix.

Quant quality was measured. In our other sessions, RABIT-KV's TPOT overhead over BF16 at this prompt length ranged from $+24\%$ to $+35\%$.

6.3 Scaling with Context and Concurrency

Concurrency. Capacity raises the load that can be admitted. With 8192-token prompts, BF16 reaches its allocator ceiling at 47 in-flight requests in all three trials and never attains an offered concurrency of 64, whereas RABIT-KV sustains 64 (Table 2). Admission is not throughput, however. RABIT-KV completes fewer requests per second than BF16 at every concurrency we tested. At 8192 tokens its throughput is $0.88\times$ that of BF16 at C=1 and $0.37\times$ at C=32, and it falls in absolute terms beyond C=16 (1.424, 0.870, and 0.323 req/s at C=16, 32, and 64). At 2048 tokens the ratio declines from $0.70\times$ at C=1 to $0.12\times$ at C=64.

Context length. For single requests, RABIT-KV's TPOT overhead over BF16 stays between $+31\%$ and $+39\%$ for prompts from 512 to 16,384 tokens. The 32,736-token point is different. The prompt no longer fits one prefill chunk, the remaining 16,352 tokens take the later-chunk path of Section 4, and TTFT rises to 114.4 s against 4.4 s for BF16. This point is the nominal model limit of our configuration, not a demonstrated maximum context. We do not interpret the lower RABIT-KV TTFT observed at 8K and 16K (supplementary appendix) as evidence of a general prefill advantage: the trend reverses sharply once the later-chunk path is exercised at 32K.

Where the time goes. Diagnostic profiling points to serialized per-request and per-token processing, and to closed-page work in long-context prefill, as the major bottleneck families of the implementation. The profiler itself slowed RABIT-KV execution several-fold, so we use it only qualitatively: its component shares are not performance measurements, and all numbers in this section come from unprofiled runs.

6.4 Quality Evaluation

The results in Table 3 are logical measurements of the representation, not outputs of the serving engine. As described in Section 5, the prompt is prefilled densely in BF16, its keys and values are converted to the RABIT-KV state, and every later token follows the residual, open-group, and closed-page lifecycle. In continuation perplexity the first scored token of each window comes from the BF16 prefill; in long-context generation all but the last prompt token are prefilled and the last one is processed on the converted state.

Continuation perplexity. On Llama-3.1-8B, perplexity rises from 7.515 to 7.633, a relative increase of 1.56% with a paired 95% confidence interval of $+1.12\%$ to $+2.05\%$ (Table 3). The loss is small but systematic: 30 of the 32 windows are worse under RABIT-KV.

Long-context tasks. RABIT-KV matches BF16 on all tested NIAH and Passage Retrieval cases: 57 of 57 at context lengths of 4,096, 8,192, and 16,384 tokens, and a score of 100.0 on 200 retrieval examples, with no individual score changing. BF16 is saturated on both sets, so these tasks cannot resolve small differences. On HotpotQA, RABIT-KV scores 58.27 versus 59.30 for BF16, a difference of -1.03 F1; the paired 95% confidence interval $[-4.24, +2.20]$ spans zero. Of the 100 scores, 92 are identical, five are worse, and three are better. The interval does not establish equivalence at this sample size.

Model	Task	N	BF16	RABIT-KV	Difference [paired 95% CI]
Llama-3.1-8B	WikiText-2 PPL ↓	32 windows	7.515	7.633	$+1.56\%$ $[+1.12\%, +2.05\%]$
Llama-3.1-8B	NIAH correct ↑	57	57	57	0 of 57 scores change
Llama-3.1-8B	Passage Retrieval ↑	200	100.0	100.0	0.0 [0.0, 0.0]
Llama-3.1-8B	HotpotQA F1 ↑[a]	100	59.30	58.27	-1.03 $[-4.24, +2.20]$
Qwen2.5-7B	WikiText-2 PPL ↓	32 windows	6.787	112.39	$+1555.9\%$ $[+1320\%, +1828\%]$

Table 3. Logical quality of the fixed operating point under the evaluator of Section 5. BF16 and RABIT-KV run on identical inputs. [a]Primary scorer; with the secondary scorer, 59.85 against 58.71, a difference of -1.14 $[-4.30, +2.03]$.

Perplexity in the serving engine. Separately from Table 3, the supplementary appendix reports Llama-3.1-8B continuation perplexity scored from the serving engine's own logits, with BF16 pages and with the packed RABIT-KV pages, one request at a time, under a privately frozen protocol. On its 28 primary windows of the same WikiText-2 pool, packed serving raises perplexity by 1.42% when the context is prefilled in one chunk (97.5% CI $+0.86\%$ to $+2.00\%$) and by 1.96% when it is forced into two 512-token chunks, so that the second chunk attends to an already packed prefix ($+1.07\%$ to $+2.93\%$). These are end-to-end differences of the engine, which also uses different dense prefill kernels for the two caches, not a quantization-only effect. The protocol was frozen before an earlier attempt and not changed afterwards; that attempt was interrupted after it had exposed the paired one-chunk results of 19 of the 28 primary windows and part of a twentieth, but no two-chunk result.

6.5 Cross-Model Behavior

The representation and its capacity transfer to the second model; the quality of the fixed operating point does not. For Qwen2.5-7B the engine builds the same packed pages and allocates $5.28\times$ the BF16 capacity (Section 6.1). Serving runs through the exact general fallback paths instead of the Llama-specialized kernels: median TPOT is 48.70 ms against 26.96 ms for BF16 ($+80.7\%$), a figure that characterizes the fallback paths and not the specialized ones. At the same operating point, Qwen2.5-7B continuation perplexity rises from 6.787 to 112.39 ($+1555.9\%$, paired 95% confidence interval $+1320\%$ to $+1828\%$), with all 32 windows worse. We did not run long-context tasks for Qwen2.5-7B under this evaluator. A post-hoc diagnostic outside the registered protocol, also in the logical evaluator (supplementary appendix), finds that excluding the zero padding of the open key group from its quantization range removes most of the excess loss, and that on eight windows the key error which remains lies in the closed pages rather than the open group. A replication of both findings on 32 windows of the WikiText-2 validation split, specified and privately frozen before scoring, met its pre-specified criteria; earlier use of that split cannot be excluded, so it is not held-out validation. Neither diagnostic changes the results above, and the serving engine still uses the frozen policy. A single frozen low-bit operating point is not quality-robust across the two tested models.

7 Discussion and Limitations

Capacity is not throughput. RABIT-KV substantially increases allocator-visible KV capacity, and in the workloads we tested this did not yield higher throughput. Capacity determines what can be admitted; how fast admitted requests complete is determined by the execution paths. The current implementation prioritizes correctness and physical packing over cross-request and cross-token kernel batching, and it serializes substantial work per request and per token. Diagnostic profiling points to this serialized processing, and to closed-page work in long-context prefill, as the major bottleneck families. The profiler heavily perturbed execution, so its component shares are not precise performance measurements and we report none of them. We do not claim that these costs are intrinsic to the representation; batched cross-request and cross-token execution remains future work.

Quality is model-dependent. At the fixed operating point, Llama-3.1-8B retains most of its quality and Qwen2.5-7B does not. The representation and its capacity transfer between the two geometries; the quality of a quantization policy does not follow from that. The operating point was deliberately not retuned per model. The Qwen2.5-7B degradation is localized only in the logical evaluator, post hoc and in a privately frozen replication on the WikiText-2 validation split (supplementary appendix). A pre-specified ablation, also only in the logical evaluator, finds that once padding is excluded, exact group parameters remove most of the remaining excess log-perplexity of Qwen2.5-7B over BF16, but little of the excess while padding is included: the two effects interact and are not separate, additive contributions. For Llama-3.1-8B the ablation detects no effect of the 8-bit group parameters. Neither change has been made in the serving engine. No repaired policy has been validated in the serving engine or on held-out inputs. The results cover two models and should not be extrapolated to others in either direction.

Scope of the quality evaluation. Quality is measured logically, with an evaluator that reproduces the stored-cache representation and its aging but uses different execution machinery from the serving engine. Serving outputs were scored only for Llama-3.1-8B continuation perplexity, one request at a time, on an engine instrumented to verify packed page writes and reads (Section 6.4). That measurement covers prefill in one chunk and in two forced chunks, in which new prompt tokens attend to an already packed prefix, but not prompts beyond the per-step token budget, concurrent requests sharing it, or generation tasks; quality on those paths, which are taken at the 32K point and under concurrency, is unmeasured. Long-context quality was measured for Llama-3.1-8B only, and its outputs are short, so it mainly tests the converted prompt state: a generated token reaches a closed page in 4 of the 100 HotpotQA examples and in no NIAH or Passage Retrieval example (supplementary appendix). NIAH and Passage Retrieval are saturated for BF16 on the tested sets and cannot resolve small differences. HotpotQA uses 100 examples and perplexity uses 32 windows; a confidence interval that spans zero does not establish equivalence. We measured no quality for FP8 or TurboQuant, so the capacity comparison with them is not a quality-matched one.

Scope of the design space. All quality results concern one operating point, K3 / V2 / G32 / R4 / META8g64. Of its components, only the metadata precision was ablated with the validated evaluator, in the logical evaluator only (supplementary appendix); the others were studied during development with an earlier evaluator and not repeated. We therefore do not claim that the individual choices are optimal, or that this point is the best trade-off for either model; the earlier ablations are mentioned only in the supplementary appendix, as history. The serving results likewise cover one GPU type and one engine snapshot, and the latency statistics are descriptive.

8 Related Work

Low-bit KV representations. KIVI (Liu et al., 2024) quantizes keys per channel and values per token, and combines 2-bit group-wise quantization (groups of 32) with a full-precision residual of recent tokens that is quantized in batches once it fills. RABIT-KV adopts these axes and the group size. KVQuant (Hooper et al., 2024) reaches sub-4-bit precision with per-channel keys quantized before the rotary embedding, non-uniform datatypes, and per-vector dense-and-sparse quantization, keeps the first token in full precision, and uses custom kernels. SKVQ (Duanmu et al., 2024) reorders channels into homogeneous groups, applies clipped dynamic quantization, keeps a sliding window of recent tokens at high precision, and stores quantization parameters in FP8 so that its reported average bits include them. AsymKV (Tao et al., 2024) shows that the output is more sensitive to key than to value quantization and assigns the two different bit-widths by layer. TurboQuant (Zandieh et al., 2025) is a data-oblivious vector quantizer based on random rotation and per-coordinate scalar quantization, evaluated on KV caches at 2.5 and 3.5 bits per channel. RABIT-KV does not claim a new quantization primitive. It differs in its operating point (3-bit keys, 2-bit values, a four-token residual), in re-quantizing the open group at every token instead of batching, and in quantizing the group parameters a second time to 8 bits. The logical diagnostics of the supplementary appendix locate most of the Qwen2.5-7B loss in the last two departures, the zero-padded open group and the 8-bit group parameters; we did not evaluate KIVI or SKVQ themselves.

Serving-system integration. vLLM (Kwon et al., 2023) introduced paged KV allocation, which RABIT-KV uses unchanged. Low-bit caches have since been built into paged engines. Upstream vLLM integrates a TurboQuant backend (vLLM contributors, 2026) that stores rotated, scalar-quantized keys and uniformly quantized values in the paged cache, sizes pages by the packed slot, offers presets with unequal key and value precision, and can keep boundary layers in full precision. Our engine snapshot is based on a later upstream revision and contains that revision's TurboQuant backend unmodified; all TurboQuant measurements in this paper refer to one preset of it and do not characterize the others. SAW-INT4 (Jia et al., 2026) treats KV quantization as a systems co-design problem: it identifies the constraints that paged layouts and fused attention kernels impose, shows that token-wise 4-bit quantization after block-diagonal Hadamard rotation satisfies them, and reports end-to-end serving throughput. It notes that a residual buffer of unquantized tokens complicates storage in uniform paged blocks; RABIT-KV stores its residual and open-group state outside the paged pool. Minima-KV (Kozyrev & Maiboroda, 2026) has a closely related tiered serving design: recent and protected pages stay in FP8, older pages move to a packed 3-bit format, format-specific kernels compute partial attention states that are merged by a global softmax, and memory is reported per live token against BF16 and FP8.

Quantized keys for token selection. RaBitQCache (Li et al., 2026) applies RaBitQ (Gao & Long, 2024), a randomly rotated 1-bit vector quantizer from nearest-neighbor search, to keys for a different purpose: it keeps the full-precision KV cache, uses the binary key codes and an INT4 query only to estimate attention scores, and attends exactly to the tokens selected by an adaptive Top-p rule. It reduces the KV data read per decoding step rather than the stored state; in its released vLLM integration the paged pool keeps the full-precision page size and the key index is held beside

it. RABIT-KV is the converse case: it replaces the stored state by low-bit codes and attends to every token, and it uses neither rotation nor binary codes. The two approaches are complementary.

RABIT-KV is therefore not the first packed or paged low-bit KV cache, nor the first to account for storage beyond nominal bits. Where SAW-INT4 targets a serving-friendly 4-bit point and Minima-KV a mixed-format hierarchy, RABIT-KV contributes the physical realization and joint measurement of one more aggressive asymmetric operating point: the capacity the allocator actually provides, in one matched session with BF16, FP8, and a TurboQuant configuration of the same engine; the latency and throughput it costs; and its quality under an evaluator checked against an independent reference, including a model on which that quality does not hold.

9 Conclusion

A low-bit KV cache is commonly described by the precision of its codes, but a serving system benefits only from the capacity its allocator can actually provide. RABIT-KV realizes one fixed low-bit operating point as packed pages in a paged serving engine, accounting for asymmetric key and value precision, residual tokens, quantized metadata, and online aging. On the evaluated H100 setup its paged block pool holds $5.28\times$ the KV tokens of BF16, $2.64\times$ those of FP8, and $1.72\times$ those of the tested TurboQuant configuration; per-sequence residual and open-group state outside the pool is accounted for separately. The comparison with FP8 and TurboQuant covers capacity and single-request latency; quality, context scaling, and concurrency were compared with BF16 only, so it is not a quality-matched comparison. That capacity has a measured cost: higher per-token latency than all three baselines, lower throughput than BF16 at every tested concurrency, and a large first-token delay beyond the first prefill chunk. Quality is retained closely on Llama-3.1-8B and degrades severely on Qwen2.5-7B at the same operating point. Physical capacity, quality, and serving cost are separate outcomes of a low-bit cache design, and they have to be evaluated jointly.

References

Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., and Li, J. LongBench: A bilingual, multitask benchmark for long context understanding. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2308.14508.

Duanmu, H., Yuan, Z., Li, X., Duan, J., Zhang, X., and Lin, D. SKVQ: Sliding-window key and value cache quantization for large language models, 2024. arXiv:2405.06219.

Gao, J. and Long, C. RaBitQ: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proceedings of the ACM on Management of Data*, 2(3):1–27, 2024. doi: 10.1145/3654970.

Hooper, C., Kim, S., Mohammadzadeh, H., Mahoney, M. W., Shao, Y. S., Keutzer, K., and Gholami, A. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2401.18079.

Jia, J., Li, J., Zhou, Z., Heo, J. H., Wang, J., Dao, T., Song, S. L., Athiwaratkun, B., Xu, C., Zhang, T., and Wu, X. SAW-INT4: System-aware 4-bit KV-cache quantization for real-world LLM serving, 2026. arXiv:2604.19157.

Kozyrev, S. and Maiboroda, D. Minima-KV: Retention-preserving KV cache compression with mixed-format paged attention, 2026. arXiv:2608.23834.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.

Li, W., Dong, J., Zhang, H., Shi, W., Lu, W., and Du, X. RaBitQCache: Rotated binary quantization for KV-Cache in long context LLM inference. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, 2026. arXiv:2606.31519.

Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., and Hu, X. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.02750.

Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models, 2016. arXiv:1609.07843.

Meta. Meta Llama 3.1 8B Instruct (model card). Released 23 July, 2024. https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct.

Tao, Q., Yu, W., and Zhou, J. AsymKV: Enabling 1-bit quantization of KV cache with layer-wise asymmetric quantization configurations, 2024. arXiv:2410.13212.

vLLM contributors. [Attention Backend] TurboQuant: 2-bit KV cache compression with 4x capacity. vLLM pull request #38479, merged 15 April, 2026.

https://github.com/vllm-project/vllm/pull/38479.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. M. HuggingFace's Transformers: State-of-the-art natural language processing, 2019. arXiv:1910.03771.

Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 technical report, 2024. arXiv:2412.15115.

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018. arXiv:1809.09600.

Zandieh, A., Daliri, M., Hadian, M., and Mirrokni, V. TurboQuant: Online vector quantization with near-optimal distortion rate, 2025. arXiv:2504.19874.

AI USAGE DISCLOSURE

Generative AI tools assisted with this work as detailed below. The author takes full responsibility for all content.

Tools. Four AI tools were used: ChatGPT, Claude Code, OpenAI Codex, and Google Deep Review. No other AI tools were used.

Ideation. The research ideas were developed through the author's discussions with ChatGPT, Claude Code, and OpenAI Codex.

Code generation. Most of the experiment harnesses, evaluation and analysis scripts, the independent reference implementation, and parts of the serving implementation were written or modified with Claude Code and Codex. Correctness is established by the checks described in the paper (bit-exact conformance against an independent reference, registered validity checks), not by the AI tools.

Experiment design and execution. Experiment protocols, validity checks, and the post-hoc diagnostic of the supplementary appendix were drafted in sessions with the AI coding tools, which also orchestrated cloud GPU runs and audited their outputs.

Serving-engine perplexity. The serving-engine perplexity measurement in the supplementary appendix was carried out in sessions with OpenAI Codex, which drafted its protocol and private freeze record, wrote its instrumentation, harness, and audit code, and launched and monitored its cloud GPU runs; Claude Code independently recomputed its results from the archived per-token scores and drafted its text in the manuscript and supplementary appendix.

Validation-split replication. The replication in the supplementary appendix was carried out in sessions with Claude Code. Claude Code drafted its pre-specified design, criteria, and private freeze record; adapted its scoring code and wrote its audit and analysis code; launched and monitored its single cloud GPU run; wrote and ran the independent audit and recomputation of its results; and drafted the replication text of the manuscript and supplementary appendix. The author authorized each step that used cloud resources. The validity of the run rests on its pre-specified controls and anchors, not on the AI tool.

Component ablation. The metadata ablation in the supplementary appendix was carried out in sessions with Claude Code, which drafted its design, validity checks, and private freeze record; adapted the scoring and audit code from the replication; ran its offline validation; launched and monitored its single cloud GPU run; wrote and ran its independent audit; and drafted its text. The author approved its scope and cost before the run. The validity of the run rests on its pre-specified controls, not on the AI tool.

Literature search. Details of related work were partly extracted from paper full texts with AI tools. The author checked every related-work claim against the original papers. For the comparison with RaBitQCache, Claude Code also read its released code, and it computed the storage decomposition in the supplementary appendix with a script from the cited storage layouts.

Writing. AI tools helped draft the text of the manuscript and supplementary appendix, which the author reviewed and revised.

Automated draft review. The author submitted complete drafts (manuscript and appendix) to Google Deep Review at two stages of revision; each returned an automated report of possible issues. With Claude Code, the author checked each point of both reports against the code, run records, and data, and adopted, partly adopted, or declined it on that basis. The resulting changes clarify scope and wording and add detail derived from existing code and records; no experiment was run in response, including those the second report suggested. The reports are AI-generated reviews, not human reviews and not independent experimental validation.

Data. No evaluation data were generated by AI tools. The needle-in-a-haystack cases are constructed programmatically from WikiText-2 text. Text produced by the evaluated models (Llama-3.1-8B-Instruct and Qwen2.5-7B-Instruct) appears only as experimental output.